UNIVERSITY OF ZULULAND

A NODE FOR AFRICAN THOUGHT

RESEARCH PROPOSAL

With the provisional title:

Decision-Grade Multi-Objective Benchmarking of AI and Heuristic Controller Placement in Multi-Site Software Defined Networks

For the degree of

In the field of Computer Sciences

FACULTY OF SCIENCE, AGRICULTURAL AND ENGINEERING

Student name: Thabang Nhlokoma Buthelezi
Student number: 230011908
Current highest qualification: BSc Honours (Computer Science)

Supervisor:
Co-Supervisor:

Year: 2026

Estimated date of submission: November 2026

TABLE OF CONTENTs

1 INTRODUCTION 2
2 BACKGROUND OF THE STUDY 2
3 PROBLEM STATEMENT AND RESEARCH QUESTIONS 2
3.1 Problem Statement 2
3.2 Research question(s) 2
4 RESEARCH AIM AND OBJECTIVES (or HYPOTHESES, where applicable) . 2
4.1 Research Aim 3
4.2 Research objectives (or Hypotheses) 3
5 CONTRIBUTION OF THE STUDY 3
6 LITERATURE REVIEW 3
7 THEORETICAL (or CONCEPTUAL) FRAMEWORK 3
8 RESEARCH METHODOLOGY 3
8.1 Research philosophy 4
8.2 Research approach 4
8.3 Research design 4
8.7 Data collection 4
8.8 Data analysis method(s) and procedure 4
9 RESEARCH QUALITY: ETHICAL AND SAFETY ISSUES 4
9.1 Research quality 4
9.2 Ethical and safety considerations 5
10 INTELLECTUAL PROPERTY 5
11 RESOURCES REQUIRED AND PROJECT PLAN 5
11.1 Resources required 5
11.2 Budget 5
11.3 Research process - Gantt chart 6
12 FEASIBILITY OF THE STUDY 7
13 DISSEMINATION PLAN 7
14 PROPOSED CHAPTER DIVISION 7
15 REFERENCES 7
16 CANDIDATE'S DECLARATION 7
17 SUPERVISOR(S) DECLARATION 7
18 ANNEXURE A: PARTICIPANT INFORMED CONSENT DECLARATION 9

1 INTRODUCTION

Software Defined Networking (SDN) is now a structural backbone for modern multi-site networks, where control is centralized but the performance of the entire fabric still depends on one unresolved bottleneck: the Controller Placement Problem (CPP). Where controllers are placed determines not only latency, but also the resilience of the control plane under failure **[Heller et al., 2012]**. In this setting, controller placement is not a minor configuration choice. It is a decision-making problem that directly shapes control-plane survivability, operational responsiveness, and the stability of inter-site communication.

That decision environment creates a Decision Crisis. Classical heuristics are attractive because they are fast, transparent, and easy to reproduce, yet they are also rigid and limited in how far they can adapt to topology variation. AI-driven methods promise greater adaptability, but they introduce a different challenge: their gains are often reported without a defensible account of Computational Cost (Seconds per Episode). The result is a decision gap between methods that are computationally cheap and methods that are potentially better, but not yet clearly justified in complexity-to-performance terms **[Farahi et al., 2026] [Benoudifa et al., 2023]**.

This study addresses that gap by treating controller placement as a multi-objective optimization problem rather than a single-metric ranking exercise. Its core contribution is not merely to compare algorithms, but to evaluate decision utility: where AI gains, if any, are large enough to justify their complexity tax. The analysis therefore combines latency, reliability, and Computational Cost (Seconds per Episode) into a unified 3-axis frame, enabling the identification of non-dominated sets rather than isolated winners. That framing is essential for distinguishing methods that are merely accurate from methods that are operationally defensible.

The research gap is clear. While AI-assisted controller placement is promising, the literature still lacks reproducible evidence across standardized topologies that resolves all three dimensions together: latency, reliability, and cost **[Farahi et al., 2026] [Benoudifa et al., 2023]**. Many studies report improvement on one axis while leaving the others under-specified. This proposal instead advances a pilot-validated, multi-objective design that is grounded in a successful proof-of-concept run of approximately 0.0226 seconds per episode, giving immediate confidence that the factorial experiment is computationally tractable on local resources.

The remainder of the proposal is structured to follow this logic. Section 6 reviews the literature and establishes the empirical gap. Section 7 formalizes the problem through MOCO, Pareto reasoning, Control Plane Survivability, and the MDP framing used for DQN/DRL policy learning. Section 8 operationalizes the methodology and experimental controls. Section 10 clarifies IP stewardship and the handling of technical artifacts, while Sections 11 to 13 move from resource feasibility to dissemination, showing how the project remains executable, reproducible, and academically publishable within the official 2026 timeline

2 BACKGROUND OF THE STUDY

2.1 From Legacy Decentralized Networking to SDN

Traditional enterprise and carrier networks were built around tightly coupled, device-centric control logic. In that legacy model, routing intelligence, forwarding behaviour, and policy enforcement were distributed across individual devices, making adaptation slow and operations difficult to standardize across sites. As networks became larger and more interdependent, this decentralized architecture became increasingly difficult to manage because every topology change demanded coordinated reconfiguration.

Software Defined Networking (SDN) emerged as the response to that complexity. Its defining feature is the separation of concerns: the Control Plane is decoupled from the Data Plane, and forwarding devices no longer need to make autonomous control decisions. Instead, a logically centralized controller acts as the network's brain, translating policy into flow rules and orchestrating behaviour across the fabric **[Heller et al., 2012]**. That decoupling improved programmability and visibility, but it also made controller placement a first-order performance variable.

2.2 Multi-Site Topologies and the Rise of the Controller Placement Problem

The CPP becomes especially important in multi-site SDN where controllers and switches are separated by meaningful geographical and topological distance. In such environments, the distance between a switch and its controller introduces propagation delay across control exchanges, path computation, and failure-response events. A placement that is acceptable in a small local network may become inefficient or fragile when applied to a geographically distributed backbone.

This issue is visible in large-scale backbones and multi-domain research settings such as Internet2 and ATT-MPLS, where the control plane must remain responsive while spanning diverse sites and heterogeneous paths. At that scale, controller placement must account for Control Plane Survivability, because failures can isolate switches from their nearest control authority and destabilize the topology **[Radam et al., 2022]**.

2.3 Evolution of Placement Logic: Manual, Heuristic, and Adaptive Search

The history of controller placement mirrors the broader evolution of network optimization. Early approaches relied on manual or deterministic placement, where engineers applied domain experience, static diagrams, and simple centrality assumptions to position controllers. These methods were straightforward, but they did not scale well and offered little adaptability when topology size, failure modes, or traffic patterns changed.

The next stage introduced Heuristic vs. Stochastic Search. Greedy rules, clustering methods, k-means, and related meta-heuristics improved runtime efficiency and often produced acceptable placements with modest computational effort. Their strength was speed; their weakness was rigidity. As the search space grew, especially in multi-site and multi-controller settings, the problem began to exhibit State-Space Explosion, and local rules alone were no longer sufficient to guarantee robust long-term performance.

Modern work has therefore shifted toward Deep Reinforcement Learning (DRL) and other adaptive agents. These methods treat placement as a sequential decision problem and attempt to learn policies that improve over time as they interact with the environment. In principle, this offers a stronger response to topology diversity and failure dynamics **[Farahi et al., 2026] [Benoudifa et al., 2023]**. In practice, adaptive learning introduces a new issue: the performance gains must be weighed against training and inference burden.

2.4 The Emerging Metric Gap and the Bridge to This Study

The background literature is rich in latency results, but far thinner in its treatment of computational burden. This omission matters because modern placement methods are not judged only by the quality of their final solution; they are also judged by the cost of producing that solution. When computational burden is ignored, the background implicitly Favors performance-at-all-costs reasoning, even when that reasoning is no longer defensible for reproducible or resource-aware research **[Radam et al., 2022]**.

This study addresses that metric gap by moving toward Efficiency-aware Optimization. It evaluates placement methods through three coupled dimensions: latency, reliability, and Computational Cost (Seconds per Episode). The pilot-validated efficiency result of approximately 0.0226 seconds per episode shows that this study is technically executable, and the later sections formalize how that empirical foundation supports the MOCO, MDP, and Pareto-based analysis presented in Sections 7, 8, and 11.

3 PROBLEM STATEMENT AND RESEARCH QUESTIONS

3.1 Problem Statement

In multi-site SDN controller placement, method claims are frequently made from single metrics (typically latency), even though deployment decisions require joint interpretation of latency, reliability, and Computational Cost (Seconds per Episode). This creates a decision-risk gap: teams can over-adopt complex AI methods without clear evidence that gains remain defensible once computational and robustness constraints are included **[Heller et al., 2012] [Radam et al., 2022]**.

3.2 Research question(s)

Primary research question:

- In multi-site synthetic SDN topologies, do AI-driven controller placement methods provide superior multi-objective decision quality compared with baseline heuristics when latency, control-plane reliability, and Computational Cost (Seconds per Episode) are evaluated jointly?

Supporting research questions:

- Under fixed topology families, node scales, and controller budgets, what is the effect size of AI methods versus baseline heuristics on average controller-switch latency and control-plane reliability?
- After accounting for Computational Cost (Seconds per Episode), which algorithms remain practically superior and Pareto-efficient for planning use cases?
- How stable are algorithm rankings across topology families and scale levels, and where do rankings shift enough to require scenario-conditioned method selection guidance?

4 RESEARCH AIM AND OBJECTIVES (or HYPOTHESES, where applicable)

4.1 Research Aim

The primary aim of this study is to quantify the performance-to-complexity trade-offs in multi-site SDN controller placement by developing a reproducible, multi-objective evaluation framework. This study seeks to determine the precise operational conditions under which PyTorch-based Deep Reinforcement Learning (DRL) agents provide a **non-dominated advantage** over classical heuristic baselines, accounting for propagation latency, survivability, and the measured "Complexity Tax" **[Heller et al., 2012; Radam et al., 2022; Farahi et al., 2026; Benoudifa et al., 2023]**.

4.2 Research objectives (or Hypotheses)

- **Implement a Reproducible Testbed:** Develop a config-driven SDN emulation environment using Mininet and Ryu that utilizes fixed random seeds (Seed 42) to ensure 100% empirical reproducibility across all trials.
- **Execute Factorial Benchmarking:** Benchmark the performance of a DQN-based placement policy against deterministic heuristics (Random, Greedy K-Center, and K-Means) across standardized topologies, including Internet2 and ATT-MPLS.
- **Quantify 3-Axis Metrics:** Evaluate each placement method through the joint metrics of Propagation Latency (ms), Control-Plane Reachability under single-link failure, and Computational Cost (measured as Seconds per Episode, validated at a pilot rate of **0.0226s**).
- **Synthesize Pareto-Efficient Guidance:** Identify Pareto-optimal placement strategies and produce scenario-conditioned selection guidance that defines the "utility threshold" for AI adoption in SDN control planes.

5 CONTRIBUTION OF THE STUDY

5.1 Theoretical Contribution

Methodological Innovation: MOCO Pipeline (1D to 3D Evaluation).
This study contributes a formal multi-objective combinatorial optimization (MOCO) pipeline that integrates Propagation Latency, control-plane reliability ($reach_{arg}$), and Computational Cost (Seconds per Episode) in one decision architecture. By replacing single-metric ranking with non-dominated 3-axis evaluation, the study resolves a known academic conflict between latency-first benchmarking and deployment-grade trade-off analysis identified in canonical CPP and Pareto-control-plane literature **[Heller et al., 2012] [Lange et al., 2015].**

Complexity Tax Theory: DRL Utility Under Operational Overhead.
The study contributes a theoretical bridge between DRL policy quality and operational overhead by defining the boundary conditions under which adaptive AI gains remain decision-grade after computational burden is priced into the objective space. This Complexity Tax framing advances SDN control-plane theory by specifying where AI utility saturates, where it dominates, and where deterministic methods remain structurally preferable.

5.2 Practical Contribution

Decision-Support Tooling: Scenario-Conditioned Selection Matrix.
The primary applied output is a scenario-conditioned selection matrix that maps topology-scale-failure contexts to the method family (DQN/DRL vs deterministic baselines) with the best Pareto-consistent operating profile. This converts algorithm comparison into planner-ready method selection rules for real infrastructure decisions.

Reproducibility Blueprint: Drift-Free Experimental Execution.
The study delivers a config-driven, seed-locked (Seed 42) testbed blueprint for drift-free experimentation, enabling repeatable benchmark execution across cohorts, including Honors researchers and practicing engineers. The pilot efficiency evidence (0.0226s0.0226s/episode) demonstrates that this blueprint is technically executable within constrained institutional compute environments.

Evidence-Based Policy Shift: From AI-Hype to Cost-Aware Optimization.
The study provides auditable evidence for governance decisions in SDN optimization programs by replacing AI-first narratives with cost-aware, threshold-based adoption criteria. In practice, this supports institutional and operational policy that funds or deploys AI only where quantified adaptive gains exceed measured complexity overhead.

6 LITERATURE REVIEW

The literature on SDN controller placement establishes a clear technical pattern: reported gains exist, but they are highly contingent on topology selection, scale, and the set of metrics used to judge success. For that reason, the present review does not treat the literature as a neutral backdrop. It treats it as the empirical basis for the Section 8 methodology, which must resolve the field's recurring problems of single-metric reporting, synthetic-only validation, and unclear cost accounting.

6.1 Comparative Performance of AI and Heuristic Controller Placement Methods

Heller et al. (2012) provide the canonical baseline for controller placement evaluation. Their Internet2-scale results establish a 50 ms latency anchor and an average 1.4x-1.7x improvement over random placement, which is why their work remains the correct reference point for any later claim of progress. That baseline is not merely historical; it is the benchmark against which later heuristics and AI methods must be judged. Classical heuristic methods, including greedy, clustering, and k-center variants, remain competitive precisely because they preserve that baseline logic: they are fast, transparent, and capable of producing acceptable placements without expensive search overhead. Bridge: this is why Section 8 retains Internet2 and ATT-MPLS corpora rather than relying on a narrower synthetic-only benchmark, since the methodology must test any claimed improvement against the canonical latency reference rather than against an easier self-generated graph family.

Farahi et al. (2026) extend the field by showing that reinforcement-learning-based placement can produce measurable gains in both quality and adaptability, including a 24% load-balancing improvement and a 25% latency reduction. Those results matter because they show that AI methods can outperform heuristic baselines on more than one axis, but they also show that performance claims must be tied to explicit training conditions and operational metrics rather than treated as generic superiority. The important implication is that AI placement should be assessed not only by final reward, but by the cost of producing that reward under stable training and evaluation procedures. Bridge: this directly motivates the Section 8 shift from vague improvement language to a multi-objective evaluation design that quantifies latency, reliability, and Computational Cost (Seconds per Episode), because DRL evidence is only decision-grade when its computational burden is made explicit.

Radam et al. (2022) reinforce the point that meta-heuristic controller placement can be effective, but only within a bounded performance envelope. Their reported 11-20 ms propagation latency and 0.88 ± 0.1 reliability benchmark show that heuristic search can produce strong placement outcomes without the training complexity of deep learning. However, those same results also reveal why a single comparison on latency is insufficient: a method may look competitive on one metric while remaining weaker on stability, reproducibility, or multi-objective balance. Bridge: this is why Section 8 does not treat meta-heuristics as a mere historical comparator, but as a full baseline family that must be evaluated alongside AI methods under the same topology and scale conditions.

6.2 Robustness Across Topology Families and Network Scales

A second and more serious limitation in the literature is that many studies rely on synthetic graphs, limited topology families, or narrow scale ranges. That constraint weakens generalisability because controller-placement behaviour changes materially as network density, path diversity, and inter-site structure change. Heller et al. (2012) remain important here because their Internet2 results derive from a realistic backbone context rather than from a trivial toy graph, and that is precisely why their findings still function as the field's canonical baseline. Bridge: Section 8 therefore commits to Internet2 and ATT-MPLS corpora, not because those topologies are convenient, but because they are necessary to test whether placement method retain their advantage outside small synthetic graphs.

The same robustness issue appears in AI-based studies. Farahi et al. (2026) show that reinforcement learning can improve controller placement, but the value of that result depends on whether the model remains stable under topology variation, link disruption, and larger search spaces. This is why failure stress-testing is not a decorative extension of the experiment; it is the methodological response to a literature that often reports only average performance under favourable settings. Bridge: Section 8 therefore incorporates reliability under single-link failure as a core measurement, because the transition from latency-only claims to reliability-quantified evaluation is required if the study is to test whether AI gains survive realistic perturbation.

Radam et al. (2022) are useful here as a counterweight to AI-centric optimism. Their results show that meta-heuristic methods can achieve good propagation latency and acceptable reliability without the training burden of deep networks, but those gains are still bound to the experimental topology and the search landscape. The point is not that heuristics are universally better; it is that their strengths are easier to reproduce when the topology is fixed and the scale is moderate. Bridge: this justifies the Section 8 factorial design, because only a structured comparison across topology family and network scale can show whether the apparent advantage of one method is stable or merely incidental.

6.3 Decision Frameworks for Multi-Objective Controller Placement

The literature increasingly acknowledges that controller placement is a multi-objective decision problem, but many papers still stop short of operationalising that insight in a reproducible way. Benoudifa et al. (2023) are representative of this pattern: their work confirms the value of learning-based placement, yet it still leaves room for stronger transparency around how performance, robustness, and computational burden are jointly reported. The consequence is that the literature often demonstrates possibility, but not decision utility. Bridge: Section 8 responds by treating performance as a three-axis problem, not a single-score problem, so that the analysis can compare methods on latency, reliability, and Computational Cost (Seconds per Episode) in the same evidentiary frame.

The importance of that shift is methodological rather than rhetorical. A placement method that looks attractive on a latency plot may still be unsuitable if its training or search cost is excessive, or if its reliability collapses under failure. Conversely, a method with modest latency improvement may still be the right choice if it is stable, inexpensive, and reproducible across scenarios. This is why Pareto-front reasoning is central to the study design: it allows the comparison to distinguish between marginally better, practically better, and non-dominated solutions. Bridge: Section 8 therefore uses Pareto analysis and factorial inference because the literature's main shortcoming is not the absence of algorithms, but the absence of a disciplined way to interpret their trade-offs.

The metric bridge is especially important. Earlier studies often describe "cost" or "runtime" in generic terms, but DRL evaluation requires a transparent operational label that can be measured consistently across runs. Computational Cost (Seconds per Episode) is therefore not arbitrary renaming; it is the correct experimental translation of training burden into a thesis-ready metric that can be linked directly to the pilot results and the factorial matrix. Bridge: this metric choice is what connects the literature to the Section 8 implementation, because it permits direct comparison between model quality and the time required to obtain that quality.

6.4 Synthesis of the Literature and Research Gap

The research gap is not a lack of AI results, but a lack of reproducible, multi-objective evidence across standardized topology families. That distinction matters. Heller et al. (2012) show what a canonical baseline looks like. Farahi et al. (2026) show that AI can improve placement quality, but only when the evaluation is precise enough to support reproducibility. Radam et al. (2022) show that strong heuristic and meta-heuristic results are possible without deep learning, but those results do not by themselves prove superiority in multi-site contexts. Benoudifa et al. (2023) reinforce that reinforcement-learning-based placement is promising, yet still in need of more explicit operational comparison.

Taken together, the literature leaves three unresolved problems. First, studies are not yet consistently comparable because they do not all use standardized topology families or equivalent failure conditions. Second, many reports remain single metric in practice even when they claim to be multi-objective. Third, computational burden is usually under-specified, making it difficult to judge whether a method's accuracy gain is worth its training cost. Bridge: Section 8 is designed to close exactly these three gaps by using Internet2 and ATT-MPLS as canonical corpora, by quantifying reliability under single-link failure, and by reporting Computational Cost (Seconds per Episode) as a transparent DRL evaluation metric.

Accordingly, the methodology is not an afterthought to the review; it is the necessary consequence of the review. The literature justifies a factorial design, a Pareto-front analysis, and a reproducible pilot-validated DQN stack because only that combination can determine whether AI-driven placement genuinely outperforms heuristics once latency, reliability, and computational burden are treated as co-equal experimental outcomes.

7 THEORETICAL (or CONCEPTUAL) FRAMEWORK

7.1 Multi-Objective Combinatorial Optimization Foundation

This study formalizes controller placement as a Multi-objective Combinatorial Optimization (MOCO) problem rather than a generic constrained decision task. Let N denote candidate network nodes and C denote controllers to be placed. The core problem is to learn or construct an optimal mapping function $f: C \rightarrow N$ that minimizes a multi-attribute objective over feasible placements. In operational terms, the framework seeks placements that jointly minimize latency and Computational Cost (Seconds per Episode), while maximizing control-plane reliability under failure.

The MOCO framing is critical because the controller-placement search space grows combinatorially with topology size and controller budget, and no single scalar objective can represent deployment-grade performance. This directly grounds Section 8's factorial design and comparative protocol: each algorithm is evaluated as a search strategy over the same combinatorial space, under the same topology and scale constraints.

7.2 Pareto Optimality and Non-Dominated Set Logic

The decision-theoretic core of the framework is Pareto Optimality. A placement is considered preferable only if it is non-dominated in the joint objective space. Accordingly, the framework uses a 3D decision frame with axes for latency, reliability, and Computational Cost (Seconds per Episode). The resulting non-dominated set defines the practically defensible frontier from which method-selection guidance is derived. Within this framework, Computational Cost (Seconds per Episode) is interpreted as the Complexity Tax paid for AI-driven gains. This concept provides the formal theoretical basis for Section 11 feasibility claims: if AI methods improve latency or survivability but require disproportionate seconds-per-episode cost, their practical superiority is weakened. If gains are achieved at an acceptable complexity tax, AI methods remain decision-grade.

7.3 Control Plane Survivability as Reliability Theory

Reliability is formalized as Control Plane Survivability, not as a static availability score. The theoretical lens is Reachability Under Failure: a placement is reliable to the extent that nodes remain controller-reachable after structured perturbations. This justifies the Section 8 use of link-failure simulation rather than static, no-failure metrics.

By grounding reliability in survivability dynamics, the framework evaluates whether a method preserves operational control when the topology is disrupted. This aligns with evidence that latency-only claims are insufficient for deployment decisions (Heller et al., 2012; Farahi et al., 2026; Radam et al., 2022; Benoudifa et al., 2023). The implication is direct: method quality must be judged on both nominal performance and post-failure reachability behaviour.

7.4 Reinforcement Learning Bridge: MDP Policy vs Heuristic Priors

The AI component is framed as a Markov Decision Process (MDP). States encode topology-aware placement context, actions correspond to controller-node assignment choices, transitions reflect environment updates under placement decisions and failure scenarios, and rewards encode the multi-objective performance signal. Under this formulation, DQN/DRL represents a policy-learning mechanism that searches for high-value placement actions over time.

Crucially, the framework does not evaluate policy quality in isolation. The learned policy is tested against Heuristic Priors: baseline methods that encode established non-learning search rules. This yields a principled comparison between adaptive policy optimization and deterministic or meta-heuristic placement logic. The framework therefore predicts Section 8's core empirical test: whether the DQN policy delivers statistically and operationally meaningful gains over heuristic priors once the full 3-axis objective, including Computational Cost (Seconds per Episode), is enforced.

7.5 Framework-to-Methodology Alignment

This hardened framework establishes the mathematical and methodological logic for Section 8. MOCO defines the optimization class, Pareto/non-dominated reasoning defines decision validity, Control Plane Survivability defines reliability theory, and MDP policy evaluation defines the AI mechanism. Together, these components justify the Section 8 implementation choices: canonical topology families, failure-aware reliability measurement, factorial analysis, and explicit computational-cost accounting in seconds per episode.

8 RESEARCH METHODOLOGY

This study adopts a methodology designed to produce objective, reproducible, and decision-relevant evidence on the comparative performance of AI-driven and heuristic controller placement methods in multi-site SDN environments.

8.1 Research philosophy

This study is grounded in a positivist research philosophy. Positivism is appropriate because the phenomenon under investigation is measurable and testable: controller placement methods produce observable outcomes such as latency, reliability, and Computational Cost (Seconds per Episode). The aim is not to interpret subjective experiences, but to evaluate whether one algorithmic class performs better than another under controlled conditions.

8.2 Research approach

The study follows a deductive research approach. Deduction is suitable because the investigation begins with existing ideas from optimization theory and controller placement literature, then tests whether predefined methods satisfy explicit performance criteria. This aligns with the research trajectory: problem statement identifies a decision gap, research questions ask whether AI methods offer meaningful advantages, and the methodology tests that claim through controlled experimentation.

8.3 Research design

The study uses an experimental research design with a factorial structure. Experimental design is appropriate because the core purpose is to compare method performance under controlled and repeatable conditions. The factorial framework examines multiple independent variables simultaneously: method type, topology family, and network scale. Method type is instantiated as heuristic priors (random, greedy k-center, and k-means), meta-heuristic search (genetic algorithm), and policy-learning AI (DQN/DRL). This enables observation of main effects and interaction effects, supporting scenario-conditioned method selection guidance.

8.3.1 Factorial Matrix and Controls

The full design matrix is: topology x controller budget x algorithm.

- Independent variables: topology family {Internet2, ATT-MPLS}; controller count {k = 2, 3, 5}; algorithm {DQN, Greedy, K-Center}.
- Dependent variables: Propagation Latency (ms), reachability ($Reach_{avg}$), Computational Cost (Seconds per Episode).
- Control variables: fixed random seed (Seed 42), fixed hardware/runtime environment (same CPU/RAM host and pinned software stack), identical traffic-generation profile per cell. This produces scenario-specific evidence without mixing incomparable topology or budget regimes.

8.3.2 DQN Policy Architecture (State, Action, Reward)

To make the AI mechanism auditable and reproducible, the DQN agent is specified as follows.

- State space: topology encoded as a graph-derived feature tensor combining adjacency and link-latency structure, plus current partial controller-placement mask and remaining budget.
- Action space: discrete node index selection for the next controller placement step, with invalid-action masking to prevent duplicate placements and to enforce the target k.
- Reward logic: multi-objective penalty reward $$ R = -(\alpha \cdot L + \beta \cdot (1 - Reach_{avg})) $$ where L is normalized latency and $Reach_{avg}$ is mean post-failure reachability; higher reward is achieved by simultaneously lowering latency and improving survivability.

8.4 Data collection

Data collection is conducted in a controlled simulation and emulation workflow rather than through human participants. This is appropriate because the research problem concerns measurable algorithmic performance under standardized conditions. The collection process is designed to preserve repeatability, minimize uncontrolled variance, and support transparent cross-method comparison.

8.4.1 Parameter Justification and Technical Stack Defence

This study adopts Mininet, a PyTorch-based DQN/DRL implementation **[Farahi et al., 2026]**, and Internet2/ATT-MPLS-class topologies as a tightly coupled methodological stack because this is the only configuration that simultaneously satisfies baseline comparability, metric transparency, and operational realism across the three method families in the literature. This choice is necessitated by the empirical structure of prior work.

Group A foundational studies show that controller-placement claims are highly topology-dependent, with Internet2-scale evaluation establishing the canonical latency baseline (Heller et al., 2012). Group B meta-heuristic studies report strong gains under simulation-specific settings, but with transferability and parameter-sensitivity limitations (Radam et al., 2022). Group C DQN/DRL studies report substantial improvements but often under-specify reproducibility-critical implementation details and reliability operationalization (Farahi et al., 2026; Benoudifa et al., 2023).

First, baseline parity is non-negotiable. The use of Internet2 and ATT-MPLS-class topologies is deliberate to replicate the structural conditions under which foundational CPP latency claims were established (Heller et al., 2012). Ensuring cross-study parity at the topology layer prevents inflated AI claims caused by easier synthetic graph regimes.

Second, this study introduces a formal reliability specification to eliminate black-box scoring. Reliability is explicitly defined as the mean fraction of nodes that remain controller-reachable under single-link failure across all edge removals. Directly addressing reporting limitations where reliability is presented as an aggregate score without fully auditable operationalization (Benoudifa et al., 2023), this definition makes each reliability claim independently recalculable from graph state and controller set.

Third, the reproducibility pivot to a full PyTorch DQN/DRL stack is required by convergence-control expectations in AP-DQN and MuZero literature (Farahi et al., 2026; Benoudifa et al., 2023). The prior lightweight reward baseline is useful for exploratory benchmarking but cannot provide replay-based stabilization, target-network synchronization, and gradient-level diagnostics expected in contemporary DRL evidence. Farahi et al. (2026) report concrete DQN settings: learning rate 0.001, batch size 32, discount factor gamma 0.99, and 1000 episodes. These values provide a quantitative reference band for implementation and hyperparameter calibration.

Finally, operational validity requires Mininet with Iperf3-driven traffic generation. Analytical or offline optimization remains essential for theory but insufficient for deployment-facing inference. Mininet introduces packet-level behaviours, contention effects, and emulation-time variability that abstract graph optimization cannot represent.

8.4.2 Design and Description of Instruments

The study instruments include Mininet and Python-based simulation/emulation environment, DQN/DRL and baseline controller-placement implementations, experiment orchestration scripts, structured logging pipelines, and metric extraction utilities. This design is justified because it combines parameter control, repeatability, and operational safety, and avoids disruption of production network infrastructure.

8.4.3 Data Collection Procedures

Data collection proceeds in a controlled sequence: first, instantiate each factorial cell (topology, k, algorithm); second, execute under identical run controls; third, persist structured logs and metric artifacts for auditability. Latency is derived from control-plane communication propagation. Reliability uses an exhaustive N-1 failure engine: for each topology edge $e \in E$, remove e, recompute controller reachability, then aggregate across all $|E|$ single-link removals to obtain $Reach_{avg}$. Computational cost is reported as Seconds per Episode.

8.5 Data analysis method(s) and procedure

The analysis combines descriptive, inferential, and multi-objective methods. Descriptive statistics report means and standard deviation per factorial cell. Inferential analysis applies normality checks (Shapiro-Wilk on residuals; Q-Q inspection) before parametric tests. Factorial effects are tested via ANOVA (method x topology x k), and where omnibus effects are significant, post-hoc pairwise contrasts are resolved with Tukey's HSD. Pairwise DQN-vs-heuristic tests use t-tests when assumptions hold, with non-parametric fallback (Mann-Whitney U / Kruskal-Wallis) when violated. Pareto-front synthesis then identifies non-dominated sets across latency, $Reach_{avg}$, and Seconds per Episode. This satisfies positivist validity by making claims measurable, reproducible, and statistically decision ready.

8.5.1 Golden Thread Mapping: Research Questions to Analysis

- RQ1 (multi-objective superiority): addressed through descriptive statistics, pairwise tests, and effect-size comparisons on latency and reliability, with Computational Cost (Seconds per Episode) included as the third axis.
- RQ2 (trade-offs and practical superiority): addressed through Pareto-front analysis and non-dominated set identification across latency, reliability, and Computational Cost (Seconds per Episode).
- RQ3 (stability across topologies and scales): addressed through factorial ANOVA interaction analysis (method x topology family x network scale) and scenario-stratified ranking stability checks.

9 RESEARCH QUALITY: ETHICAL AND SAFETY ISSUES

9.1 Research quality

Research quality is operationalized through triangulation, reproducibility, and internal-validity control. Triangulation is achieved by testing across Internet2 and ATT-MPLS. Reproducibility is enforced through a config-driven pipeline (`experiment_config.json`) with fixed random seeding (`seed = 42`). Internal validity is strengthened through factorial design that isolates method, topology, and scale effects.

9.2 Ethical and safety considerations

9.2.1 Informed consent

This study does not involve human participants, survey respondents, or interviewees. It is an algorithmic evaluation conducted entirely within simulated network environments. As a result, traditional informed consent procedures do not apply.

9.2.2 Debriefing

Debriefing is not applicable because no human participants are exposed to the study procedures. The outputs of the study are simulation results, performance metrics, and comparative analyses of algorithms.

9.2.3 Withdrawal from investigation

Participant withdrawal does not apply because no participants are recruited. However, experimental runs may be stopped, repeated, or discarded if they are found to be corrupted, unstable, or methodologically invalid.

9.2.4 Protection of participants

The study is ethical because it is conducted in a sandboxed simulation environment, such as Mininet and Python-based experimental scripts, and therefore poses no risk to live university network infrastructure, operational systems, or human well-being. The work is limited to controlled computational experiments and does not interfere with production services or expose real users to harm.

9.2.5 Confidentiality and anonymity

Confidentiality concerns are minimal because the study will use only public or synthetic topology data and will not require access to sensitive institutional network configurations. No private network identifiers, user data, or operational logs from live systems will be collected or disclosed. This ensures that the research remains ethically appropriate and does not compromise institutional confidentiality.

9.2.6 Data storage

Research data, experimental logs, scripts, and generated outputs will be stored securely in password-protected local environments during active analysis. Backup copies will be maintained in encrypted cloud storage approved by the University, such as OneDrive or Google Drive where available. Version control will be managed through a private GitHub or Bitbucket repository to preserve a clear audit trail of code and analysis changes. Access to all files will be restricted to the researcher and, where required, the supervisor. This storage plan supports data integrity, traceability, and responsible research practice.

Overall, this study is ethical because it adheres to the principles of research integrity, transparency, and computational safety. It involves no human participants, does not expose sensitive systems, and is designed to generate reproducible evidence within a controlled and low-risk technical environment.

10 INTELLECTUAL PROPERTY

The intellectual property arising from this study belongs to the University of Zululand in accordance with institutional policy **[University of Zululand, n.d.]**. For the ai-sdn-controller-placement project, the protected research artifact scope includes the proposal text, the DQN implementation, trained DQN model weights, topology-aware reward functions, automated experimental runner scripts, configuration files, simulation outputs, and derived analysis artifacts. `experiment_config.json` and `pilot_metrics.json` form part of the University's intellectual corpus because they encode the reproducible experimental conditions and validated pilot evidence.

This IP position exists alongside open-source dependencies and reference assets, including PyTorch, Mininet, Ryu, and the Internet Topology Zoo. The project logic remains the University's authored work, while the underlying third-party tools continue to be governed by BSD, Apache, MIT, and related permissive licenses.

Version control serves as the primary audit trail for intellectual-property creation, attribution, and revision history. The Git-based repository records the evolution of the research artifacts, the timing of changes, and the relationship between code, configuration, and outputs, supporting academic accountability and reproducibility.

Research Data Management (RDM) governs the storage and handling of metrics, simulation logs, and exported result files. The RDM approach keeps data traceable, securely stored, and recoverable for validation, consistent with the hardened methodology and pilot-validated pipeline in Sections 8 and 11.

11 RESOURCES REQUIRED AND PROJECT PLAN

11.1 Resources required

The study uses an isolated Python 3.10+ virtual environment to preserve dependency stability, prevent version drift, and maintain the reproducibility established in Section 9.2. The software stack comprises PyTorch for the DQN implementation, Mininet for network emulation, Ryu for SDN control logic, and the custom ai-sdn-controller-placement framework. The data corpora are the Internet Topology Zoo-aligned canonical topologies used throughout the study, specifically Internet2 and ATT-MPLS. Local x86_64 CPU and RAM resources are sufficient for all training, emulation, logging, and analysis tasks because the pipeline is optimized for CPU-based DRL throughput.

11.2 Budget

As the project relies on open-source assets, the primary budget item is Computational Resource Allocation. The governing efficiency metric is Computational Cost (Seconds per Episode). Using the pilot value of 0.0226 seconds per episode, the full factorial matrix of 5 algorithms × 10 topologies × 1000 episodes = 50,000 episodes requires approximately 1,130 seconds of CPU time, or about 18.8 minutes in total. This low compute burden supports multiple stability trials and sensitivity checks while remaining comfortably below the risk threshold for the department deadlines.

11.3 Research process - Gantt chart

(below is an example – please delete and develop your own)

Table 1: xxxxxx

No.	Milestones and Dates	May 2022	August 2022	September 2022	October 2022	November 2022	Jan - March 2023	April- May 2023	June - July 2023	August 2023	September 2023	October 2023	November 2023
1	*Proposal development*												
2	*Submit proposal*												
3	*Receive feedback and Corrections*												
4	*Submit a proposal for review*												
5	*Receive feedback and Corrections*												
6	*Present proposal to department /faculty*												
7	*HDC approval*												
8	*Thesis / Dissertation development*												
	- *Further Literature review*												
	- *Conceptual framework*												
	- *Data collection*												
	- *Data analysis/ methodology*												
	- *Discussion, conclusion & recommendations*												
	- *Thesis submission for examination*												

12 FEASIBILITY OF THE STUDY

12.1 Researcher Competency and Technical Capacity

The researcher holds a foundation in BSc Computer Science and Mathematics (Undergraduate), which directly enables the theoretical and implementation competencies required for this Honors-level study. The mathematical background supports the MDP formulation (Section 7.4), the Pareto optimality reasoning (Section 7.2), and the statistical convergence analysis required to interpret DRL training dynamics. The computer science foundation enables the implementation of PyTorch-based DQN/DRL, the configuration of Mininet emulation environments, and the automation of the factorial experimental pipeline. Beyond academic grounding, the researcher has professional experience as an Introductory Programming Tutor, which ensures code quality, robust error handling, and reproducible pipeline design. This domain expertise directly translates to the code-review and documentation standards that support the reproducibility requirements in Section 9.1, the config-driven architecture that enables experiment-state resumability, and the Git-based audit trail that Section 10 mandates for IP stewardship. The combination of mathematical rigor and hands-on programming proficiency positions the researcher to execute both theoretical interpretation (statistical inference, Pareto ranking) and operational implementation (DRL training loops, metric extraction) with confidence.

12.2 Resources: Validation, Not Assumption

This study is not resource-contingent on external dependencies. The feasibility evidence is grounded in pilot validation, not on theoretical capability claims.

Infrastructure Validation: The project has been pilot-validated on a local x86_64 CPU testbed. Section 11.2 establishes the Computational Stewardship budget: using the empirically derived pilot value of 0.0226 seconds per episode, the full factorial matrix of 5 algorithms × 10 topologies × 1000 episodes = 50,000 episodes requires approximately 1,130 seconds of CPU time, or about 18.8 minutes total. This has been measured, not estimated. The local testbed is therefore not merely "available"; it is proven capable of executing the full experimental matrix within the risk-acceptable time frame. No external GPU clusters, cloud compute quotas, or third-party resource requests are required.

Self-Contained Nature: The project is operationally self-contained. No proprietary software licenses are required—all tools (PyTorch, Mininet, Ryu) are open-source under permissive licenses. No human participants are involved, so the research sidesteps clinical-ethics complexity and external review dependencies. The topology data comes from publicly available Internet Topology Zoo-aligned corpora, not from proprietary network telemetry. This means Section 3 ethics clearance follows the standard institutional research-integrity pathway, not an extended regulatory review.

12.3 Methodological Control and Risk Mitigation

The "Complexity Tax" concept from Section 7.2 operationalizes the core risk of this study: that AI methods may improve latency or reliability gains at a computational cost that undermines their practical superiority. This risk is not avoided; it is quantified and controlled.

The config-driven architecture (Section 8.4.2 and documented in `experiment_config.json`) materializes this control. Every experimental

parameter—random seed, learning rate, batch size, discount factor, topology selection, scale, and algorithm choice—is version-controlled in a single configuration artifact. This ensures that experiments can be paused, resumed, or re-executed without loss of integrity. If a run is interrupted, the checkpoint-save mechanism preserves the training state, and the experiment can resume from the last completed episode. This design eliminates the risk of lost computational work and supports transparent error recovery.

Reproducibility is further hardened through the factorial structure itself. The design matrix (Section 8.3) specifies precisely which algorithm-topology-scale combinations are tested and in what order. No ad-hoc method selection or scope creep occurs because the experimental frame is locked. This is risk-mitigation by design, not by hope.

12.4 Timeline Confidence and Project Status

The official 2026 milestone roadmap (Section 11.3) charts nine phases from February through November. The critical feasibility indicator is **Phase 1 Status: Pilot Validation is 100% Complete**. This is not a projection; the 50-episode DQN proof-of-concept has executed successfully, capturing the 0.0226 seconds-per-episode metric that anchors all downstream feasibility claims.

By the timing of this proposal (April 11, 2026), Phase 1 completion places the project ahead of the typical Honors risk curve. In most Honors timelines, pilot work is deferred until mid-project or treated as parallel exploratory activity. Here, it is already validated. This means Phases 2–9 operate with known technical risk and quantified compute requirements, not with speculative assumptions.

The Gantt chart (Section 11.3) maps the remaining eight phases to official department deadlines, from the current Phase 2 (Proposal, Apr 13–20) through Phase 9 (Final Submission, Nov 09–13). Each phase phase has documented entry and exit criteria, and each depends on demonstrable artifacts (proposal acceptance, ethics approval, data ingest completion, draft synthesis, mock exam feedback, prototype demo findings). The timeline is aggressive but not speculative: it is built on the foundation that Phase 1 pilot validation succeeded.

12.5 Closing the Loop: Golden Thread of Feasibility

This study closes the loop on all prior sections. Literature review (Section 6) identified the decision gap. Theory (Section 7) formalized MOCO, Pareto optimality, and reliability dynamics. Methodology (Section 8) specified the experimental protocol. Quality and ethics (Section 9) operationalized reproducibility and safety. IP (Section 10) clarified institutional ownership and open-source boundaries. Resources (Section 11) quantified the compute budget based on pilot evidence.

Feasibility (Section 12) binds all these together through the assertion: the researcher is competent, the resources are validated, the methods are controlled, the timeline is

grounded in completed pilot work, and the project is achievable within departmental constraints. No feasibility claims rests on unvalidated assumptions. Each assertion is anchored to empirical evidence (pilot results), technical documentation (config-driven architecture), or institutional commitment (timeline milestones).

This is not an optimistic feasibility statement. It is a data-backed proof of feasibility. The study is ready to proceed.

13 DISSEMINATION PLAN

13.1 Research Manuscript and Academic Publication

The official 2026 research timeline (Section 11.3, Phase 9: Nov 09–13) requires formal submission of a Research Paper as a core deliverable of the Honors program. This research will be documented in a high-quality manuscript prepared for peer-reviewed submission to a venue appropriate to the international SDN and network optimization community, such as SATNAC (Southern African Telecommunications Networks and Applications Conference), SAICSIT (South African Institute of Computer Scientists and Information Technologists), or a relevant international conference in the areas of software-defined networking, optimization, or network architecture.

The manuscript will feature the empirical research contributions of this study, focusing on the 3-axis Pareto trade-offs discovered during the factorial benchmarking experiments: (1) latency reduction measured in milliseconds of propagation delay, (2) control-plane reliability quantified as mean fraction of nodes reachable under single-link failure, and (3) Computational Cost (Seconds per Episode) as a transparent measure of training and search burden. The paper will serve broader audiences by resolving a critical literature gap: the lack of multi-objective, reproducible comparative evidence on when AI-driven controller placement genuinely outperforms heuristic baselines, and under what operational constraints.

By documenting the trade-off surfaces and method-selection guidance derived from the analysis, the manuscript transforms experimental results into actionable knowledge for practitioners and researchers managing multi-site SDN deployments. The publication timeline aligns with Phase 8 (Exam, Nov 04–06), allowing prototype demonstration findings to inform the final manuscript before submission within the Phase 9 window.

13.2 Technical Artifacts and Repository Dissemination

The GitHub repository (ai-sdn-controller-placement) serves as the primary long-term dissemination platform for the study. This repository encodes all technical artifacts necessary for independent reproduction and extension of the research: the experiment orchestration scripts, the PyTorch-based DQN/DRL implementation, curated configuration files, and analysis pipelines.

The reproducibility foundation is the `experiment_config.json` artifact, which documents the complete factorial matrix specification, hyperparameter settings (learning rate 0.001, batch size 32, discount factor gamma 0.99, replay memory 10,000), and random seed (seed = 42). This configuration is version-controlled and auditable, allowing any researcher to re-execute the full 50,000-episode factorial matrix and verify the pilot-validated efficiency metric of approximately 0.0226 seconds per episode on equivalent x86_64 CPU hardware.

Additionally, trained model weights and checkpoint artifacts will be documented and made available in tagged releases, enabling other researchers to directly inspect the learned DQN policy, validate the placements it generates, and apply the policy to novel topologies. This approach aligns with Section 10's IP stewardship framework: while the University of Zululand retains ownership of the research artifacts and the authored code, the dissemination strategy prioritizes transparency and reproducibility through open documentation and version-controlled artifact release, balancing institutional IP protection with the scientific imperative for independent verification.

13.3 Prototype Demonstration and Live System Validation

The official project timeline includes a critical dissemination milestone: the Prototype Demonstration (Section 11.3, Phase 8: Nov 04–06). This technical presentation provides a "visual proof" of the optimization results by demonstrating the DQN-placed controller environment in a live Mininet/Ryu software-defined network emulation. The prototype shows real-time network behavior under the learned placement policy, illustrating latency improvements, reliability under failure injection, and the computational cost of the DQN agent's decision-making in a concrete, observable context.

This hands-on demonstration serves multiple dissemination purposes: (1) it validates that theoretical benchmarks translate into operational reality, (2) it provides an engaging medium for communicating research findings to academic and technical audiences, (3) it demonstrates implementation mastery and system thinking aligned with Honors-level expectations, and (4) it creates a reproducible artifact that can be re-deployed and modified by other researchers or practitioners seeking to build on the work.

13.4 Peer and Departmental Knowledge Sharing

Immediate dissemination within the University of Zululand community occurs through the Honors Research Symposium and Faculty of Science, Agriculture, and Engineering departmental seminars. These venues allow the researcher to present findings to peer Honors students, faculty supervisors, and research groups, securing formative feedback and establishing credibility within the local research ecosystem.

The dissemination strategy in this phase emphasizes clarity of communication, technical accuracy, and pedagogical value: explaining not only what the results are, but why the methodology matters for the broader SDN control-plane literature and what implications the trade-off surfaces hold for practitioners. This contributes to the research culture at UNIZULU by modelling rigorous, multi-objective thinking about algorithmic trade-offs and demonstrating the importance of computational burden as a decision criterion alongside performance metrics.

13.5 Long-Term Stewardship and Research Legacy

Post-submission, the research artifacts remain available through the GitHub repository and institutional archival systems, ensuring long-term accessibility and supporting the reproducibility requirements of the Honors program. The configuration of the repository (with clear README files, documented hyperparameters, and tagged experiment versions) enables future researchers to cite, replicate, and extend the work.

This dissemination plan fulfils the institutional requirement for a research manuscript while maintaining alignment with the IP boundaries established in Section 10. The result is a multi-layered dissemination strategy: peer-reviewed manuscript publication, open repository architecture with documented artifacts, live technical demonstration, regional academic sharing, and institutional archival—collectively ensuring that the research reaches relevant audiences and contributes to the scientific community's understanding of multi-objective controller placement optimization in SDN environments.

14 PROPOSED CHAPTER DIVISION

Chapter 1: Introduction and Background.

Chapter 2: Literature Review and Conceptual Framework.

Chapter 3: Research Methodology (Design, Instruments, and Procedures).

Chapter 4: Presentation of Results and Data Analysis.

Chapter 5: Summary, Conclusion, and Recommendations.

15 REFERENCES

16 Benoudifa, M., Siad, L. and Belmokaddem, M. (2023) 'Autonomous solution for controller placement problem of software-defined networking using MuZero based intelligent agents', Journal of King Saud University - Computer and Information Sciences, 35(10), article 101842. doi:

- 10.1016/j.jksuci.2023.101842
- Farahi, I., et al. (2026) 'AP-DQN: A novel approach for controller placement in software-defined networks using deep reinforcement learning', Results in Engineering, 29, article 109631. doi: 10.1016/j.rineng.2026.109631
- Heller, B., Sherwood, R. and McKeown, N. (2012) 'The controller placement problem', Proceedings of the 1st Workshop on Hot Topics in Software Defined Networks (HotSDN), pp. 7-12. doi: 10.1145/2342441.2342444 URL: https://doi.org/10.1145/2342441.2342444
- Radam, N.S., Faraj Al-Janabi, S.T. and Jasim, K.Sh. (2022) 'Multi-Controllers Placement Optimization in SDN by the Hybrid HSA-PSO Algorithm', Computers, 11(8), p. 111. doi: 10.3390/computers11080111 URL: https://doi.org/10.3390/computers11080111
- University of Zululand (n.d.) Intellectual Property Policy.